

EE1102 : Introduction to Programming
Aug-Nov 2023 - Assignment 1

Things to Know for this Assignment

All writing and reading requires including `stdio.h`:

```
#include<stdio.h> /* this is a comment line (can be on separate line too) */
```

Basic structure of a C program (call it `foo.c`):

```
#include....  
int main(){ /* start of the main function. All execution starts here */  
    .... /* code */  
    return (...) /* return value to the operating system */  
} /* end of the main program */
```

To compile the code: `gcc foo.c -o foo` compiles the program **foo.c** and creates the executable **foo**.

Most programs we write will also require `math.h`, which is the include file for the `libmath.so` library. The 'so' indicates that the library is a "shared" library, which means that a single copy of the library function is in the memory which is used by all programs that require it.

To compile such a program, we have to teach the compiler that we need `libmath.so`. For that we execute: `gcc foo.c -lm -o foo`. (Actually the current gcc on my computer seems to use `libmath.so` automatically. So the `-lm` may not be required.)

To write something, use `printf("...")`.

```
printf("This is a line\n");  
This is a line
```

To print multi-line output

```
printf("This is\na line\n"); /* each \n starts a new line */  
This is  
a line
```

Note: if you had written

```
printf("This is\n a line\n");
```

A space would have been added ahead of the 'a' in the second line.

To declare an integer variable and assign it a value

```
int a;  
a=4;
```

Can also assign the value in the same line:

```
int a=4;
```

To write something that includes an integer variable (printf conversion specification for integer is %d)

```
printf("The value of a = %d\n",a); /* output printed, with %d replaced by value */  
The value of a = 4
```

Note that the following is allowed: %6d - print as integer, using 6 spaces

```
int a=8, b=120;  
printf("The value of a = %3d and b = %3d\n",8,15); /* use 2 digits for all output */  
The value of a = 8 and b = 12                      /* note extra space for 8 */
```

Addition of variables and constants

```
int b=10;  
int c=15; /* can give the variable a value inside the declaration itself */  
int d,e;  
d=a+b+c; /* addition of variables */  
e=a+4+c; /* addition of variables and constants (here 4 is a constant) */
```

To declare a string variable, and assign it a value

```
char name[ ] = "Abc Xyz";
```

The syntax will be explained later, but this is a simple way to declare string constants. To print some strings, use

```
printf("My name is %s, and my home is in %s\n", name, city);
```

%s is the symbol used to indicate a string. It will be taken from the list of variables following the format string. Again, we can modify the output of the string:

```
printf("My name is %10s, and my home is in %10s\n", name, city);
```

This limits the number of spaces to be used for name and city to be a maximum of 10. If the string has 10 spaces or less, it will print out correctly with additional spaces as required. But if the string is larger, C will print out the whole string so that the output is readable.

To declare a real number, and assign it a value

```
float a = 4.5;
```

This creates a real number that uses 32 bits of storage. The mantissa is 24 bits in size including sign, and the exponent is 8 bits in size. Note that exponent cannot exceed 7 bits as there is an offset. To print float numbers use

```
printf("I am %f cm tall\n", 66.5);
```

Note that the value for height can also be taken from a constant

To declare a long int variable. This is a little strange. "int" is not a fixed size. It depends on the cpu of the computer. So a long integer variable is called "long long int"

```
long long int aa = 123456789;  
printf("aa = %lld\n", aa);
```

This creates an integer that uses 64 bits of storage. Such an integer can store much larger size integer values. For example,

```
long long int amnt=1000; /* amount in crores to be printed out */
Int crore = 10000000; /* Number of Rs in 1 crore */
amnt = amnt*crore; /* amnt converted to Rs. */
printf("1000 crores is %lld Rs.", amt );
```

To declare a long real number, and assign it a value

```
double b = 3.5e250
```

This creates large real numbers whose mantissa or exponent will not fit into a float variable.

New additions to this section:

If we want the user to enter some information, we use the scanf function. The format is

```
scanf("format",&arg1, &arg2, ..., &argn);
```

For example, (assuming #include<stdio.h> is present at the top)

```
Int a,b;
printf("enter a and b: "); /* help message */
scanf("%d %d", &a, &b);
print("a = %d, b = %d \n",a,b);
```

The '&' symbol is needed to give scanf not the variable, but the memory location where the variable exists. Printf takes the value present at a memory. So the variable name is all that is needed. But here, we need the memory location so that scanf can write the value entered by the user into that memory.

Additional operations:

We have already looked at +, -, * and /. An additional operator is the modulus operator. $x\%y$ is defined to be the remainder when x is divided by y . Both / and % are machine dependent when the arguments are negative.

Relational operators are operators that act on logical expressions to get logical answers.

If $i=4$ and $j=7$, then

$i < j$ is True

$2*i < j$ is false.

$(i < j) \ \&\& \ (2*i > j)$ is True as both statements are true

These will be very important once we talk about control statements. In this assignment, we need them in Problem 1. When checking for the different cases of the discriminant, we need the following:

```
if( D >= 0.0 ){
    if( D == 0.0 ){
        printf("The roots are identical \n");
        ... code to find the roots' values
    } else {
        printf("The roots are real and distinct\n");
        ... code to find the roots' values
    }
} else {
    printf("The roots are complex and distinct\n");
    ... code to find the roots
}
```

We will learn more about this later, but use this for now in the assignment.

Assignment

Problem 1. Write a C program called quad.c that reads in the coefficients of a quadratic function, $p = ax^2 + bx + c$, and solves for the roots of the function. The roots may be real and different, real and same, or complex and different. You need to compute the discriminant, $D = b^2 - 4ac$. If D is positive, the roots are real. If D is zero, the roots are real and same, and if D is negative, the roots are complex. Determine the type of zeros, and compute the zeros and write out the answer as follows:

D is _____. Hence the roots are _____. The two roots are _____ and _____ respectively.

Note: You can use the sqrt function in the libmath.so library. Add `#include<math.h>` and that makes the following available:

`float sqrtf(float x);`

`Double sqrt(double x);`

Problem 2. Write a C program called polyadd.c and polymull.c that, given the coefficients of two polynomials that are first order, will calculate the sum and product of the polynomials. Note that we have not introduced for and while loops, so you may not use those. You must use scanf to read in the coefficients, compute the resulting polynomial and then print out the answer as:

The sum of _____ and _____ = _____

The product of _____ and _____ = _____

Example: If the polynomials are $3x + 5$ and $4x - 22$, the answer should be

The sum of $3x + 5$ and $4x - 22 = 7x - 17$

The product of $3x + 5$ and $4x - 22 = 12x^2 - 31x + 20$

Problem 3. Modify Problem 2 to enable your program to calculate the product of two 1st order polynomials **in two variables** and print out the result. So if the polynomials are $4a + 10b$ and $5a + 3b$, the answer should be

The product of $4a + 10b$ and $5a + 3b = 20a^2 + 62ab + 30b^2$